
Pengenalan Algoritma, Komputer, dan Representasi Data

1.1 Komputer: Alat bantu pemecahan masalah fisis

Komputer merupakan salah satu alat kerja utama dalam sains dan rekayasa. Dalam Teknik Fisika, komputer dapat digunakan untuk membaca data sensor, mengolah hasil eksperimen, menyelesaikan persamaan, menjalankan simulasi, mengendalikan perangkat, membuat visualisasi, sampai menganalisis data dalam jumlah besar.

Sebagai contoh sederhana, misalkan sebuah eksperimen menghasilkan sejumlah data temperatur:

$$T_1, T_2, \dots, T_N.$$

Kita ingin mengetahui temperatur rata-rata,

$$\bar{T} = \frac{1}{N} \sum_{i=1}^N T_i = \frac{1}{N} (T_1 + T_2 + \dots + T_N).$$

Bagi manusia yang sudah memahami matematika, persamaan tersebut tampak sederhana. Akan tetapi, komputer tidak secara langsung memahami simbol

$$\sum_{i=1}^N T_i.$$

Komputer membutuhkan serangkaian instruksi yang jauh lebih eksplisit.

Kita dapat mengubah perhitungan tersebut menjadi langkah-langkah berikut.

1. Siapkan sebuah variabel untuk menyimpan jumlah temperatur.
2. Berikan nilai awal nol pada variabel tersebut.
3. Baca temperatur pertama.
4. Tambahkan temperatur tersebut ke jumlah.
5. Baca temperatur berikutnya.
6. Tambahkan lagi ke jumlah.

7. Ulangi proses sampai seluruh data selesai dibaca.
8. Bagi jumlah tersebut dengan banyaknya data.
9. Tampilkan hasilnya.

Urutan langkah yang terdefinisi dengan jelas seperti ini disebut sebagai **algoritma**.

Pemrograman pada dasarnya merupakan proses **menerjemahkan algoritma** ke dalam **bahasa** tertentu yang kemudian diterjemahkan lebih lanjut menjadi **instruksi mesin** dalam komputer.

Sepanjang mata kuliah ini, hubungan berikut akan kerap kita gunakan:

masalah $\rightarrow$ model $\rightarrow$ algoritma $\rightarrow$ program $\rightarrow$ hasil

Untuk mahasiswa dan praktisi Teknik Fisika, bagian pertama (**pernyataan masalah**) dari alur tersebut sangat penting. Kita biasanya tidak memulai dari program. Kita memulai dari suatu persoalan fisika atau rekayasa.

Sebagai contoh, energi kinetik sebuah benda diberikan oleh

$$E_k = \frac{1}{2}mv^2.$$

Jika massa m dan kecepatan v diketahui, kita dapat menyusun prosedur:

1. baca nilai massa m ;
2. baca nilai kecepatan v ;
3. hitung v^2 ;
4. kalikan dengan m ;
5. kalikan hasilnya dengan $1/2$;
6. tampilkan energi kinetik.

Dalam bahasa C, bagian perhitungannya dapat ditulis sebagai

```
energy = 0.5 * mass * velocity * velocity;
```

Dalam Python, algoritma yang sama dapat ditulis sebagai

```
energy = 0.5 * mass * velocity**2
```

Kedua program tersebut memiliki sintaksis yang tampak berbeda, tetapi ide komputasinya sama. Contoh ini membawa kita pada perbedaan penting antara **algoritma** dan **program**.

Algoritma adalah prosedur penyelesaian masalah, sedangkan **program** adalah implementasi algoritma menggunakan suatu bahasa pemrograman.

Dengan demikian, aktivitas belajar pemrograman itu bukanlah semata-mata mempelajari sintaks C atau Python. Aktivitas kita dalam mata kuliah ini akan sekaligus belajar merumuskan suatu persoalan menjadi langkah-langkah komputasi yang jelas, benar, dan efisien.

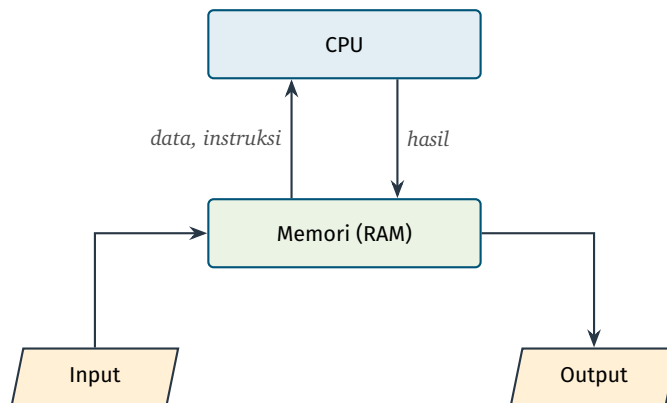
1.1.1 Apa sebenarnya yang dilakukan komputer?

Untuk memahami pemrograman dengan baik, terlebih dahulu kita perlu memiliki gambaran sederhana mengenai cara kerja komputer.

Komputer modern sangatlah kompleks. Namun, untuk keperluan awal kita dapat memandangnya sebagai sistem yang memiliki beberapa bagian penting:

- ▶ prosesor atau *central processing unit* (CPU);
- ▶ memori utama atau *random access memory* (RAM);
- ▶ media penyimpanan;
- ▶ perangkat masukan atau input;
- ▶ perangkat keluaran atau output.

Secara sederhana, kita dapat menggambarkan aliran informasi seperti ilustrasi di bawah ini.

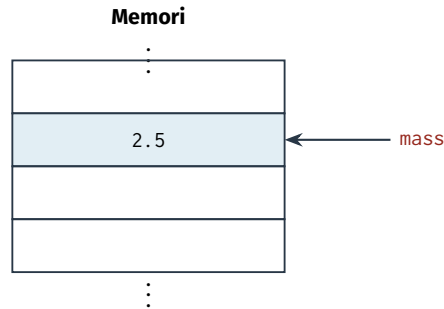


CPU menjalankan instruksi. Memori menyimpan data dan instruksi yang sedang digunakan. Media penyimpanan seperti SSD menyimpan data secara lebih permanen.

Misalkan dalam bahasa C kita menulis

```
double mass = 2.5;
```

Secara konseptual, komputer menyediakan suatu lokasi di memori untuk menyimpan nilai **2.5**. Kita dapat membayangkan penyimpanan nilai tersebut seperti ilustrasi di bawah ini.



Pada kenyataannya, komputer tidak menyimpan tulisan **2.5** seperti yang kita lihat di layar. Nilai tersebut direpresentasikan menggunakan pola bit. Konsep bagaimana data direpresentasikan dalam memori akan sangat penting ketika kita mulai mempelajari tipe data, larik (*array*), *pointer*, dan pengolahan berkas.

1.1.2 CPU dan instruksi

CPU pada komputer menjalankan sejumlah instruksi dasar. Dalam bentuk yang sangat disederhanakan, CPU dapat melakukan operasi seperti

- ▶ memindahkan data dari suatu lokasi ke lokasi lain;
- ▶ membaca nilai dari memori;
- ▶ menyimpan nilai ke memori;
- ▶ menjumlahkan dua nilai;
- ▶ mengurangi dua nilai;
- ▶ membandingkan dua nilai;
- ▶ menentukan instruksi berikutnya yang akan dijalankan.

Program yang tampak sederhana bagi kita dapat menghasilkan sejumlah besar instruksi mesin. Sebagai contoh, kode

```
energy = 0.5 * mass * velocity * velocity;
```

pada akhirnya harus diterjemahkan menjadi operasi-operasi yang dapat dijalankan oleh prosesor.

CPU tidak memahami sintaks bahasa C secara langsung sehingga diperlukan proses penerjemahan.

1.2 Pemrosesan kode sumber hingga eksekusi

Ketika menulis program dalam bahasa C (atau bahasa pemrograman lainnya), kita dikatakan menulis **kode sumber** (*source code*). Misalkan kita membuat berkas bernama **hello.c** yang berisi

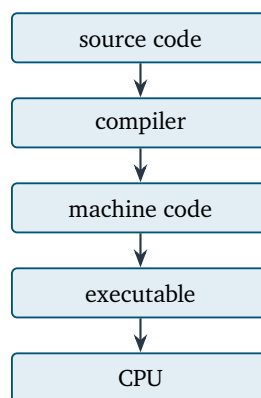
```
#include <stdio.h>
```

```
int main(void)
{
    printf("Hello, Teknik Fisika!\n");

    return 0;
}
```

Berkas tersebut dapat dibaca manusia, tetapi belum dapat langsung dijalankan oleh CPU yang menggunakan kode mesin (*machine code/instruction*). Kita masih memerlukan program lain yang disebut **compiler**.

Secara sederhana,



atau

source code $\xrightarrow{\text{compiler}}$ machine code.

Salah satu *compiler* C yang banyak digunakan adalah GCC.

1.2.1 Instalasi GCC di Windows

Ada beberapa cara untuk instalasi GCC di Windows. Di sini kita ambil pendekatan **MSYS2** sebagai cara terbaik di Windows untuk dapat digunakan bersama lingkungan pengembangan terpadu (*integrated development environment*) yang bernama **VS Code**.

Mengunduh dan Menginstal MSYS2

1. Kunjungi situs resmi MSYS2 di <https://www.msys2.org>.
2. Unduh berkas penginstal (`msys2-x86_64-...exe`).
3. Jalankan berkas penginstal tersebut dan ikuti petunjuk pada layar.
4. Gunakan direktori instalasi bawaan, yaitu `C:\msys64`.
5. Selesaikan proses instalasi sampai tuntas.

Verifikasi: Buka File Explorer dan pastikan folder `C:\msys64` telah terbentuk beserta subfoldernya.

Memasang Toolchain GCC melalui MSYS2

1. Buka terminal **MSYS2 UCRT64** dari menu Start Windows.
2. Masukkan perintah berikut untuk memperbarui basis data dan memasang *compiler* GCC, G++, *debugger* GDB, serta pustaka pendukung:

```
pacman -S --needed base-devel mingw-w64-ucrt-x86_64-toolchain
```

3. Saat muncul pertanyaan pemilihan paket (`Enter a selection (default=all)`), tekan **Enter** untuk memilih semua paket standar.
4. Saat diminta konfirmasi penginstalan (`Proceed with installation? [Y/n]`), ketik **Y** lalu tekan **Enter**.
5. Tunggu proses pengunduhan dan instalasi sampai selesai.

Verifikasi: Pada terminal MSYS2 UCRT64, jalankan perintah `gcc --version`. Terminal akan menampilkan rincian versi GCC yang terpasang.

Menambahkan Jalur GCC ke Variabel Sistem (Path) Windows

Pendaftaran jalur *compiler* ke variabel sistem bertujuan agar perintah GCC dapat dipanggil dari terminal mana pun, termasuk terminal internal VS Code.

1. Buka menu Start di Windows, ketik **Environment Variables**, lalu pilih **Edit the system environment variables**.
2. Pada jendela System Properties yang muncul, klik tombol **Environment Variables...** di bagian bawah.
3. Pada panel **System variables** (atau **User variables** jika tidak memiliki hak administrator), cari variabel bernama **Path**, pilih variabel tersebut, lalu klik **Edit...**
4. Klik tombol **New**, kemudian masukkan lokasi folder *executable* atau *binary* dari UCRT64:

```
C:\msys64\ucrt64\bin
```

5. Klik **OK** pada seluruh jendela yang terbuka untuk menyimpan konfigurasi.

Verifikasi:

- Buka jendela baru Command Prompt atau Windows PowerShell (jendela terminal lama harus ditutup terlebih dahulu agar variabel baru dimuat).
- Jalankan perintah berikut:

```
gcc --version
```

```
g++ --version
```

```
gdb --version
```

- Jika informasi versi tampil tanpa pesan kesalahan perintah tidak dikenal, konfigurasi jalur sistem telah berhasil.

Memasang Ekstensi C/C++ pada VS Code

1. Buka aplikasi VS Code. Atau unduh dan instal dulu VS Code jika belum punya ([download link](#))
2. Masuk ke menu Extensions dengan menekan tombol pintasan **Ctrl + Shift + X**.
3. Masukkan kata kunci **C/C++** pada kolom pencarian.
4. Cari ekstensi resmi bernama **C/C++** yang diterbitkan oleh **Microsoft**.
5. Klik tombol **Install** dan tunggu hingga instalasi selesai.

Verifikasi: Periksa status ekstensi pada daftar installed extensions; pastikan tombol berubah menjadi Disable atau Uninstall yang menandakan ekstensi telah aktif.

Menguji Kompilasi Program di VS Code

1. Buat sebuah folder proyek baru di komputer, misalnya **C:\LatihanC**.
2. Di VS Code, buka folder tersebut melalui menu **File > Open Folder...**
3. Buat berkas baru bernama **hello.c** di dalam folder tersebut.
4. Masukkan kode program sederhana berikut:

```
#include <stdio.h>

int main(void)
{
    printf("Hello, Teknik Fisika!\n");

    return 0;
}
```

5. Simpan berkas dengan menekan **Ctrl + S**.
6. Buka terminal terintegrasi di VS Code melalui menu **Terminal > New Terminal** atau menggunakan pintasan **Ctrl + `** (tanda aksen graf).
7. Jalankan perintah kompilasi berikut di terminal:

```
gcc hello.c -o hello.exe
```

8. Jalankan berkas biner hasil kompilasi:

```
.\hello.exe
```

- **Verifikasi:** Terminal VS Code akan menampilkan output teks **Hello, Teknik Fisika!** secara langsung tanpa kesalahan kompilasi.

Untuk saat ini kita belum perlu memahami seluruh bagian program C tersebut. Sintaks C akan dibahas secara sistematis pada pertemuan berikutnya.

Hal yang perlu dipahami sekarang adalah alurnya:

```
source code → compiler → executable → execution
```

1.2.2 Instalasi GCC di Linux

Penulis sebenarnya lebih menyarankan mahasiswa untuk melakukan pemrograman dalam sistem operasi Linux. Bagi mahasiswa yang belum berani menggunakan Linux secara langsung, dapat menginstal Windows Subsystem for Linux (WSL).

Langkah persiapan

Pastikan subsistem Linux telah aktif dan distribusi Linux terpasang di Windows. Jika belum terpasang, buka PowerShell dengan hak administrator, lalu jalankan:

```
wsl --install
```

Nyalakan ulang komputer jika diminta oleh sistem, kemudian selesaikan proses inisialisasi nama pengguna (*username*) serta kata sandi (*password*) pada distribusi Linux yang terbuka.

Hati-hati dalam tahapan input kata sandi pada sistem Linux kita tidak dapat melihat langsung apa yang kita tulis. Namun, sebetulnya setiap input huruf dari kibor kita dibaca oleh sistem sehingga kita harus yakin mengetik dengan benar.

Memasang *compiler* GCC dan *debugger* di WSL

Buka terminal distribusi Linux WSL (biasanya Ubuntu yang menjadi bawaan), lalu perbarui daftar paket dan pasang paket kompilasi:

```
sudo apt update
sudo apt install -y build-essential gdb
```

Verifikasi instalasi dengan memeriksa versi:

```
gcc --version
gdb --version
```

Terminal akan menampilkan informasi nomor versi GCC dan GDB yang terpasang di lingkungan Linux tersebut.

Memasang Ekstensi WSL di Visual Studio Code

- ▶ Buka Visual Studio Code di Windows.
- ▶ Buka menu Extensions dengan menekan pintasan keyboard **Ctrl + Shift + X**.
- ▶ Cari ekstensi bernama **WSL** yang diterbitkan oleh Microsoft.
- ▶ Klik tombol **Install**.

Membuka Lingkungan Kerja WSL di VS Code

Buka terminal distribusi WSL, buat direktori kerja baru di dalam sistem berkas Linux, lalu jalankan perintah `code`. Urutannya seperti ini:

```
mkdir -p ~/proyek-c && cd ~/proyek-c  
code .
```

Perintah ini akan mengunduh dan memasang komponen VS Code Server di dalam lingkungan Linux secara otomatis pada eksekusi pertama. Setelah selesai, jendela VS Code di Windows akan terbuka dengan indikator status di pojok kiri bawah bertuliskan nama distribusi WSL yang sedang aktif.

Proses penghubungan VS Code ke WSL juga dapat dilakukan langsung dari jendela VS Code di Windows:

- ▶ Tekan tombol **Ctrl + Shift + P** untuk membuka Command Palette.
- ▶ Ketik dan pilih perintah **WSL: Connect to WSL**.

Memasang ekstensi C/C++ pada sisi Remote WSL

Ekstensi bahasa pemrograman harus berjalan langsung di dalam lingkungan Linux WSL agar fitur analisis kode dan *debugging* dapat membaca berkas *header* sistem Linux.

- ▶ Buka kembali menu Extensions (**Ctrl + Shift + X**) pada jendela VS Code yang telah terhubung ke WSL.
- ▶ Cari ekstensi **C/C++** resmi dari Microsoft.
- ▶ Klik tombol **Install in WSL**.

Kompilasi dan eksekusi program C

Seperti contoh untuk Windows, kita dapat membuat berkas baru bernama `hello.c` pada direktori proyek:

```
#include <stdio.h>  
  
int main(void)  
{  
    printf("Hello, Teknik Fisika!\n");  
  
    return 0;  
}
```

Simpan berkas dengan menekan **Ctrl + S**.

Buka terminal terintegrasi di VS Code menggunakan pintasan keyboard **Ctrl + `** (tanda aksen graf). Terminal yang terbuka otomatis merupakan sesi shell Linux WSL.

Kompilasi kode program menggunakan GCC:

```
gcc hello.c -o hello.x
```

Jalankan berkas *executable* hasil kompilasi:

```
./hello.x
```

Teks keluaran program akan langsung dicetak pada panel terminal VS Code.

1.3 Representasi data

Sampai di sini kita dapat mengajukan pertanyaan yang lebih mendasar. Bagaimana komputer menyimpan data? Misalkan kita mempunyai nilai 42 atau -17 atau 3.1415926535. Bagaimana nilai-nilai tersebut disimpan di dalam memori? Untuk menjawabnya kita perlu mengenal **bit**.

Bit merupakan satuan informasi yang memiliki dua kemungkinan keadaan, yaitu 0 atau 1. Secara elektronik, kedua keadaan ini dapat direpresentasikan menggunakan dua rentang kondisi fisik yang berbeda. Detail implementasi elektroniknya tidak diperlukan dalam mata kuliah ini. Yang penting bagi kita adalah bahwa data pada tingkat rendah direpresentasikan menggunakan kombinasi nol dan satu.

Delapan bit dapat membentuk satu **byte**,

$$1 \text{ byte} = 8 \text{ bit.}$$

Contoh satu byte adalah **01001101**.

Karena setiap bit mempunyai dua kemungkinan nilai, delapan bit mempunyai

$$2^8 = 256$$

kombinasi berbeda. Prinsip sederhana ini akan menjelaskan mengapa tipe data komputer mempunyai rentang nilai yang terbatas.

1.3.1 Sistem bilangan desimal dan biner

Dalam kehidupan sehari-hari kita menggunakan sistem bilangan desimal atau basis sepuluh. Sebagai contoh, 572 sebenarnya merupakan singkatan dari

$$5(10^2) + 7(10^1) + 2(10^0).$$

Kita tahu

$$10^2 = 100, \quad 10^1 = 10, \quad 10^0 = 1,$$

sehingga

$$572 = 5(100) + 7(10) + 2(1).$$

Sistem biner bekerja dengan prinsip yang sama, tetapi menggunakan basis dua. Digit yang tersedia hanya 0 dan 1. Misalkan kita mempunyai bilangan biner 1101_2 . Subskrip 2 menunjukkan bahwa bilangan tersebut menggunakan basis dua. Nilainya adalah

$$1101_2 = 1(2^3) + 1(2^2) + 0(2^1) + 1(2^0).$$

Karena

$$2^3 = 8, \quad 2^2 = 4, \quad 2^1 = 2, \quad 2^0 = 1,$$

kita dapat tuliskan

$$1101_2 = 8 + 4 + 0 + 1 = 13_{10}.$$

Jadi,

$$1101_2 = 13_{10}$$

Contoh lainnya adalah 10110_2 . Kita memperoleh

$$10110_2 = 1(2^4) + 0(2^3) + 1(2^2) + 1(2^1) + 0(2^0),$$

sehingga

$$10110_2 = 16 + 0 + 4 + 2 + 0 = 22_{10}.$$

Mengubah desimal menjadi biner

Untuk mengubah bilangan desimal bulat menjadi biner, salah satu metode yang dapat digunakan adalah pembagian berulang dengan dua. Misalkan kita ingin mengubah 13_{10} menjadi bilangan biner. Kita mulai dengan

$$13 = 6(2) + 1.$$

Kemudian

$$6 = 3(2) + 0,$$

$$3 = 1(2) + 1,$$

dan

$$1 = 0(2) + 1.$$

Sisa pembagian adalah

$$1, \quad 0, \quad 1, \quad 1.$$

Jika dibaca dari bawah ke atas, 1101 . Jadi,

$$13_{10} = 1101_2.$$

Sebagai latihan, coba tunjukkan bahwa

$$42_{10} = 101010_2.$$

Perhatikan pola berikut:

$$42 = 32 + 8 + 2 = 2^5 + 2^3 + 2^1.$$

Karena itu, bit pada posisi 5, 3, dan 1 bernilai satu:

$$42_{10} = 101010_2.$$

1.3.2 Sistem heksadesimal

Perhatikan bahwa bilangan biner cepat menjadi panjang. Sebagai contoh,

```
1111011010110010
```

cukup sulit dibaca oleh manusia. Dengan alasan ini, dalam komputasi sering digunakan sistem bilangan **heksadesimal**, yaitu sistem berbasis enam belas.

Digit heksadesimal adalah 0 1 2 3 4 5 6 7 8 9 A B C D E F dengan

$$\begin{aligned} A &= 10, & B &= 11, & C &= 12, \\ D &= 13, & E &= 14, & F &= 15. \end{aligned}$$

Alasan heksadesimal sangat berguna dalam komputasi adalah

$$16 = 2^4.$$

Artinya, satu digit heksadesimal tepat merepresentasikan empat bit. Sebagai contoh,

$$0000_2 = 0_{16},$$

$$0001_2 = 1_{16},$$

$$1010_2 = A_{16},$$

dan

$$1111_2 = F_{16}.$$

Sekarang perhatikan bilangan 10101111_2 . Kita dapat mengelompokkannya menjadi $1010 \ 1111$. Kelompok pertama adalah

$$1010_2 = A_{16},$$

sedangkan kelompok kedua adalah

$$1111_2 = F_{16}.$$

Jadi,

$$10101111_2 = AF_{16}$$

Notasi heksadesimal akan muncul kembali ketika kita membahas representasi data dan alamat memori.

Berapa banyak nilai yang dapat disimpan oleh sejumlah bit?

Jika terdapat satu bit, jumlah pola yang mungkin adalah 2. Jika terdapat dua bit,

$$2^2 = 4.$$

Pola yang mungkin adalah

```
00
01
10
11
```

Untuk tiga bit,

$$2^3 = 8.$$

Secara umum, dengan n bit terdapat

$$2^n$$

pola bit yang berbeda. Hasil ini tampak sederhana, tetapi sangat penting. Dengan delapan bit,

$$2^8 = 256$$

pola dapat direpresentasikan. Dengan enam belas bit,

$$2^{16} = 65536.$$

Dengan 32 bit,

$$2^{32} = 4294967296.$$

Jumlah pola ini kemudian dapat dipetakan ke nilai-nilai yang ingin kita representasikan.

1.3.3 Representasi Bilangan Bulat

Unsigned integer

Mari mulai dari kasus termudah, yaitu bilangan bulat (*integer*) tidak negatif. Jika delapan bit digunakan untuk menyimpan sebuah *unsigned integer*, pola terkecil adalah **00000000** yang mewakili 0. Sementara itu, pola terbesar adalah **11111111** yang bernilai

$$2^7 + 2^6 + \dots + 2^1 + 2^0.$$

Nilai eksaknya adalah 255. Dengan demikian, rentang *unsigned integer* delapan bit adalah

$$0 \leq x \leq 255.$$

Secara umum, *unsigned integer* dengan n bit mempunyai rentang

$$0 \leq x \leq 2^n - 1.$$

Untuk 16 bit,

$$0 \leq x \leq 65535.$$

Untuk 32 bit,

$$0 \leq x \leq 4294967295.$$

Penting untuk diperhatikan bahwa jumlah bit terbatas menyebabkan rentang nilai juga terbatas.

Bagaimana dengan bilangan negatif?

Kita juga membutuhkan representasi untuk bilangan seperti

$$-1, \quad -27, \quad -100.$$

Komputer modern umumnya menggunakan representasi yang disebut **two's complement** untuk *signed integer*.

Pada tahap ini kita belum perlu membahas seluruh detail *two's complement*. Hal yang perlu diketahui adalah bahwa untuk *signed integer n-bit*, rentang tipikalnya adalah

$$-2^{n-1} \leq x \leq 2^{n-1} - 1.$$

Untuk delapan bit,

$$-128 \leq x \leq 127.$$

Untuk 16 bit,

$$-32768 \leq x \leq 32767.$$

Perhatikan bahwa rentangnya tidak simetris sempurna. Ada satu bilangan negatif tambahan. Kita akan kembali ke detail representasi data ketika mempelajari tipe data dalam C.

Overflow

Jumlah bit yang terbatas membuat sebuah integer tidak dapat bertambah tanpa batas.

Mari kita tinjau *unsigned integer* delapan bit. Nilai maksimum yang dapat direpresentasikan adalah 255. Secara matematis,

$$255 + 1 = 256.$$

Namun, 256 tidak dapat direpresentasikan oleh delapan bit *unsigned* karena

$$256 = 2^8.$$

Representasi binernya membutuhkan sembilan bit:

$$256_{10} = 100000000_2.$$

Ketika hasil suatu operasi keluar dari rentang representasi tipe data, kita menghadapi kondisi yang disebut **overflow**.

Overflow merupakan salah satu contoh awal bahwa matematika dan komputasi tidak selalu identik. Dalam matematika, bilangan bulat dapat sebesar apa pun. Dalam komputer, bilangan tersebut disimpan menggunakan jumlah bit yang terbatas. Jadi, ketika melakukan komputasi, kita harus selalu mempertimbangkan **nilai matematis** dan **representasi komputer**. Keduanya berkaitan erat, tetapi tidak sama.

1.3.4 Karakter juga direpresentasikan sebagai bilangan

Komputer tidak hanya menyimpan angka, tetapi juga teks. Misalnya, **A** atau **Teknik Fisika**.

Bagaimana komputer menyimpan huruf atau karakter (*character*) semacam itu? Salah satu idenya adalah memberikan suatu bilangan kepada setiap karakter. Dalam ASCII, misalnya, karakter **A** diasosiasikan dengan nilai desimal 65. Karakter **B** diasosiasikan dengan 66. Karakter **a** diasosiasikan dengan 97. Bilangan-bilangan tersebut pada akhirnya direpresentasikan menggunakan bit sehingga teks juga dapat disimpan dalam memori komputer. Secara konseptual,

$$\text{karakter} \longrightarrow \text{kode numerik} \longrightarrow \text{bit}.$$

ASCII hanya mencakup kumpulan karakter yang relatif terbatas. Sistem modern menggunakan Unicode untuk merepresentasikan kumpulan karakter yang jauh lebih luas. Kita tidak perlu mempelajari detail Unicode sekarang. Poin pentingnya adalah bahwa hampir semua informasi digital akhirnya perlu direpresentasikan menggunakan pola bit.

1.3.5 Representasi bilangan real

Sejauh ini, bilangan bulat relatif mudah dipahami. Masalah menjadi lebih rumit ketika kita ingin menyimpan bilangan *real* seperti

$$3.141592653589793 \dots$$

atau

$$\frac{1}{3} = 0.333333333333333 \dots$$

atau

$$6.02214076 \times 10^{23}.$$

Komputer memiliki jumlah bit yang terbatas, sedangkan beberapa bilangan membutuhkan jumlah digit yang tidak terbatas untuk direpresentasikan secara eksak. Untuk menyimpan bilangan real, komputer biasanya menggunakan representasi titik kambang (**floating-point**). Idenya mirip dengan notasi ilmiah. Dalam notasi ilmiah desimal, kita dapat menulis

$$602200000000000000000000$$

sebagai 6.022×10^{23} . Secara umum, kita dapat menuliskan bentuk semacam

$$x = \pm m \times 10^e.$$

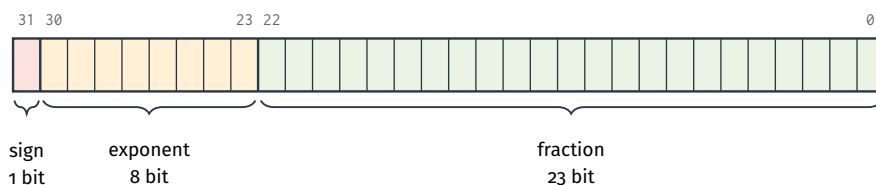
Representasi *floating-point* menggunakan gagasan yang serupa, tetapi komputer bekerja menggunakan basis dua. Secara konseptual, bilangan dapat ditulis dalam bentuk

$$x = (-1)^s m 2^e,$$

dengan

- ▶ s menyatakan tanda;
- ▶ m menyatakan bagian signifikan;
- ▶ e menyatakan eksponen.

Salah satu standar yang paling banyak digunakan untuk representasi *floating-point* adalah **IEEE 754**. Untuk tipe *single precision* 32-bit, susunan bit secara umum adalah



atau

```
s eeeeeeee ffffffffffffffffffffffff
```

Untuk bilangan normal pada format 32-bit, nilainya dapat dipahami melalui bentuk

$$x = (-1)^s (1.f) 2^{E-127},$$

dengan E merupakan nilai bilangan dari delapan bit eksponen. Bilangan 127 disebut **bias** untuk eksponen pada format *single precision*. Bagian **1.f** menunjukkan bahwa untuk bilangan normal, bit **1** di depan titik biner tidak perlu disimpan secara eksplisit. Bit yang disimpan pada bagian *fraction* hanyalah bagian setelah titik biner.

Contoh representasi 5.75 dalam IEEE 754

Sebagai contoh, kita ingin merepresentasikan 5.75 menggunakan format IEEE 754 *single precision*. Pertama, kita ubah bagian bulat ke biner:

$$5_{10} = 101_2.$$

Bagian pecahannya adalah

$$0.75 = 0.5 + 0.25,$$

sehingga

$$0.75_{10} = 0.11_2.$$

Dengan demikian,

$$5.75_{10} = 101.11_2.$$

Selanjutnya kita tuliskan dalam bentuk biner ternormalisasi,

$$101.11_2 = 1.0111_2 \times 2^2.$$

Dari bentuk tersebut kita memperoleh tiga bagian yang diperlukan.

Bit tanda

Karena 5.75 bernilai positif,

$$s = 0.$$

Jadi, bit tandanya adalah **0**.

Eksponen

Eksponen sebenarnya adalah

$$e = 2.$$

Pada format *single precision*, eksponen disimpan menggunakan bias 127,

$$E = e + 127 = 2 + 127 = 129.$$

Bilangan 129 dalam biner adalah

$$129_{10} = 10000001_2.$$

Jadi, bagian eksponennya adalah **10000001**.

Fraction

Dari bentuk ternormalisasi $1.0111_2 \times 2^2$, angka **1** paling depan tidak disimpan secara eksplisit. Bagian setelah titik biner adalah **0111**. Kemudian, kita tambahkan nol hingga panjangnya menjadi 23 bit,

```
01110000000000000000000
```

Dengan demikian, ketiga bagian IEEE 754 adalah

```
sign      = 0
exponent  = 10000001
fraction  = 01110000000000000000000
```

Jika digabungkan,

```
0 10000001 01110000000000000000000
```

atau tanpa spasi,

```
0100000010111000000000000000000
```

Representasi tersebut adalah representasi IEEE 754 *single precision* untuk nilai 5.75.

Dalam bentuk heksadesimal, pola bit yang sama dapat ditulis sebagai **0x40B80000**. Contoh ini memperlihatkan bahwa representasi *floating-point* pada dasarnya menyimpan tiga informasi:

tanda + eksponen + bagian signifikan.

Untuk 5.75, representasinya dapat dilakukan secara eksak karena pecahan tersebut memiliki representasi biner yang berhingga. Namun, tidak semua bilangan desimal mempunyai sifat tersebut.

Bilangan seperti 0.1 tidak mempunyai representasi biner berhingga. Alasannya, jumlah bit pada bagian *fraction* terbatas, sehingga komputer harus menyimpan nilai yang sangat dekat dengan 0.1, tetapi bukan nilai matematis 0.1 secara eksak. Konsekuensi inilah yang akan kita bahas pada bagian berikutnya.

Mengapa 0.1 sulit direpresentasikan?

Dalam sistem desimal, pecahan

$$\frac{1}{3}$$

tidak dapat direpresentasikan menggunakan jumlah digit yang terbatas:

$$\frac{1}{3} = 0.3333333333 \dots$$

Jika kita hanya mempunyai beberapa digit, kita harus melakukan pendekatan, misalnya 0.3333. Hal yang serupa terjadi pada sistem biner.

Bilangan desimal 0.1 tidak mempunyai representasi biner berhingga. Representasinya terus berlanjut. Karena komputer hanya menyediakan jumlah bit yang terbatas, komputer harus menyimpan nilai biner yang sangat dekat dengan 0.1, bukan nilai 0.1 matematis secara eksak. Hal yang sama terjadi untuk banyak bilangan desimal lainnya. Akibatnya, $0.1 + 0.2$ dapat menghasilkan representasi internal yang sedikit berbeda dari 0.3.

Dalam beberapa sistem pemrograman kita dapat melihat hasil seperti

```
0.30000000000000004
```

Hasil ini bukan karena komputer tidak mampu melakukan penjumlahan sederhana. Masalahnya berada pada representasi bilangan. Nilai 0.1 dan 0.2 yang digunakan dalam operasi tersebut sendiri sudah merupakan pendekatan.

Kesalahan pembulatan

Perbedaan kecil yang muncul akibat representasi terbatas disebut **rounding error** atau kesalahan pembulatan. Untuk satu operasi sederhana, kesalahan tersebut biasanya sangat kecil. Namun, dalam komputasi ilmiah kita dapat melakukan jutaan atau bahkan miliaran operasi. Dalam algoritma tertentu, kesalahan kecil dapat terakumulasi atau diperbesar.

Dari uraian ini, mahasiswa Teknik Fisika perlu memahami sejak awal bahwa hasil komputasi numerik perlu dianalisis, bukan hanya diterima begitu saja. Misalkan teori memberikan nilai x_{exact} dan program menghasilkan x_{computed} . Kita sering lebih tertarik pada kesalahannya,

$$\Delta x = x_{\text{computed}} - x_{\text{exact}}.$$

Kita juga dapat melihat kesalahan absolut

$$|\Delta x| = |x_{\text{computed}} - x_{\text{exact}}|.$$

Dalam praktik pemrograman, dua nilai *floating-point* sering dibandingkan menggunakan suatu toleransi ε :

$$|x_1 - x_2| < \varepsilon.$$

Ide ini akan menjadi sangat penting ketika kita mempelajari komputasi numerik lebih lanjut, misalnya dalam mata kuliah fisika komputasi atau teknik komputasi.

1.3.6 Presisi (*precision*) dan rentang (*range*)

Dua istilah dalam representasi bilangan yang perlu dibedakan adalah presisi (**precision**) dan rentang (**range**). *Precision* berkaitan dengan banyaknya informasi signifikan yang dapat disimpan, sementara *range* berkaitan dengan seberapa besar atau kecil nilai yang dapat direpresentasikan.

Sebagai ilustrasi sederhana, suatu sistem bilangan dapat mempunyai *range* yang sangat besar tetapi *precision* yang terbatas. Dalam C kita akan mengenal tipe seperti `float` dan `double`. Keduanya merepresentasikan bilangan *floating-point*, tetapi umumnya `double` menggunakan lebih banyak bit dan memberikan *precision* lebih tinggi. Dalam banyak komputasi ilmiah pada mata kuliah ini, kita akan lebih sering menggunakan tipe `double` untuk bilangan real.

1.4 Program C pertama untuk perhitungan fisika

Sekarang kita dapat menggabungkan beberapa gagasan yang telah dipelajari. Misalkan sebuah benda mempunyai massa

$$m = 2.0 \text{ kg}$$

dan besar kecepatan

$$v = 3.0 \text{ m/s.}$$

Energi kinetiknya adalah

$$E_k = \frac{1}{2}mv^2.$$

Secara hitungan manual,

$$E_k = \frac{1}{2}(2.0)(3.0)^2.$$

Perhatikan

$$3^2 = 9,$$

sehingga

$$E_k = \frac{1}{2}(2)(9) = 9 \text{ J.}$$

Sekarang kita implementasikan perhitungan tersebut dalam bahasa C.

```
#include <stdio.h>

int main(void)
{
    double mass = 2.0;
    double velocity = 3.0;

    double energy = 0.5 * mass * velocity * velocity;

    printf("Kinetic energy = %f J\n", energy);

    return 0;
}
```

Simpan program tersebut sebagai

```
kinetic_energy.c
```

Kemudian kompilasi menggunakan

```
gcc -Wall -Wextra kinetic_energy.c -o kinetic_energy
```

dan jalankan

```
./kinetic_energy
```

Kita memperoleh keluaran seperti

```
Kinetic energy = 9.000000 J
```

Perhatikan bahwa program menggunakan

```
double mass = 2.0;
```

untuk menyimpan massa. Baris

```
double velocity = 3.0;
```

menyimpan kecepatan. Kemudian

```
double energy = 0.5 * mass * velocity * velocity;
```

melakukan perhitungan

$$E_k = \frac{1}{2}mv^2.$$

Untuk sementara kita belum perlu menghafalkan semua aturan sintaksis tersebut. Kita akan membahas variabel, tipe data, operator, dan ekspresi dengan lebih terstruktur pada pertemuan-pertemuan berikutnya.

1.4.1 Program komputer perlu diperiksa

Misalkan program berhasil dikompilasi. Apakah itu berarti program pasti benar? Jawabannya jelas tidak.

Perhatikan program berikut.

```
#include <stdio.h>

int main(void)
{
    double mass = 2.0;
    double velocity = 3.0;

    double energy = 0.5 * mass * velocity;

    printf("Kinetic energy = %f J\n", energy);

    return 0;
}
```

Program ini kemungkinan besar dapat dikompilasi tanpa masalah. Namun, persamaan yang digunakan adalah

$$E = \frac{1}{2}mv,$$

bukan

$$E_k = \frac{1}{2}mv^2.$$

Program akan berjalan, tetapi hasilnya salah. Situasi ini adalah contoh sederhana dari galat logika (**logic error**). Oleh karena itu, terdapat perbedaan besar antara

program dapat dijalankan

dan

program menghasilkan jawaban yang benar.

Dalam sains dan rekayasa, kita selalu perlu memeriksa apakah hasil program masuk akal. Salah satu cara paling sederhana adalah membandingkan hasil program dengan kasus yang dapat dihitung secara manual. Untuk contoh tadi, kita sudah mengetahui bahwa

$$m = 2, \quad v = 3$$

seharusnya menghasilkan

$$E_k = 9 \text{ J.}$$

Jika program menghasilkan nilai lain, kita mengetahui ada sesuatu yang perlu diperiksa. Kebiasaan ini sangat penting:

prediksi → jalankan → bandingkan

1.4.2 Galat sintaksis (*syntax error*)

Jenis kesalahan lain adalah galat sintaksis. Misalkan kita menulis

```
printf("Hello")
```

padahal bahasa C mengharuskan tanda titik koma:

```
printf("Hello");
```

Compiler akan “protes” dan mendeteksi kesalahan seperti ini.

Secara sederhana, kita dapat membedakan dua bentuk kesalahan.

- ▶ **Syntax error** terjadi ketika kode tidak mengikuti aturan bahasa sehingga *compiler* tidak dapat menerjemahkannya dengan benar.
- ▶ **Logic error** terjadi ketika program secara sintaks benar tetapi algoritma atau perhitungannya salah.

Nantinya kita juga akan bertemu berbagai jenis kesalahan lain.

1.4.3 Peringatan (*warning*) dari *compiler*

Selain mendeteksi kesalahan, *compiler* dapat memberikan peringatan (**warning**). *Warning* berbeda dengan *error* karena *error* biasanya menyebabkan proses kompilasi gagal, sedangkan *warning* sering kali masih memungkinkan program dikompilasi, tetapi *compiler* menemukan sesuatu yang perlu diperhatikan.

Sepanjang mata kuliah kita akan membiasakan diri menggunakan

```
gcc -Wall -Wextra program.c -o program
```

dan membaca pesan *compiler* dengan hati-hati. Opsi `-Wall` dan `-Wextra` meminta compiler memberikan lebih banyak peringatan mengenai hal-hal yang berpotensi menimbulkan masalah.

Perlu dipahami bahwa pesan *error* dan *warning* bukanlah musuh *programmer*. Pesan tersebut justru memberikan informasi mengenai bagian program yang mungkin perlu diperiksa.

1.4.4 Debugging

Proses mencari dan memperbaiki kesalahan dalam program disebut **debugging**.

Pada tahap awal, *debugging* dapat dilakukan dengan beberapa kebiasaan sederhana:

- ▶ baca pesan *compiler* dengan teliti;
- ▶ periksa kembali persamaan yang digunakan;
- ▶ uji program menggunakan input yang sederhana;
- ▶ hitung hasil secara manual jika memungkinkan;
- ▶ periksa nilai variabel;
- ▶ ubah satu hal pada satu waktu;
- ▶ jangan langsung menganggap komputer yang salah.

Cara-cara *debugging* yang lebih canggih dan sistematis dapat dipelajari melalui berbagai tutorial pemrograman dari aspek yang lebih teknis. Untuk mata kuliah ini, kita cukup melakukan teknik *debugging* yang disebutkan di atas.

1.4.5 Contoh program fisika sederhana

Perhitungan hukum Ohm

Misalkan

$$V = IR.$$

Jika

$$I = 2.5 \text{ A}$$

dan

$$R = 10 \, \Omega,$$

maka

$$V = (2.5)(10) = 25 \text{ V}.$$

Program C sederhana untuk menghitungnya adalah

```
#include <stdio.h>

int main(void)
{
    double current = 2.5;
    double resistance = 10.0;
```

```

double voltage = current * resistance;

printf("Voltage = %f V\n", voltage);

return 0;
}

```

Program ini mengikuti pola umum

input → processing → output.

Inputnya adalah I dan R . Prosesnya adalah

$$V = IR.$$

Outputnya adalah V . Pola tersebut muncul hampir di seluruh aktivitas komputasi.

Gerak lurus

Untuk gerak dengan percepatan konstan,

$$x(t) = x_0 + v_0 t + \frac{1}{2} a t^2.$$

Misalkan

$$\begin{aligned} x_0 &= 2.0 \text{ m}, \\ v_0 &= 5.0 \text{ m/s}, \\ a &= 3.0 \text{ m/s}^2, \end{aligned}$$

dan

$$t = 4.0 \text{ s}.$$

Secara manual,

$$x = 2.0 + (5.0)(4.0) + \frac{1}{2}(3.0)(4.0)^2.$$

Maka

$$\begin{aligned} x &= 2 + 20 + \frac{1}{2}(3)(16), \\ x &= 22 + 24, \end{aligned}$$

dan kita peroleh

$x = 46 \text{ m}.$

Program C-nya dapat ditulis sebagai

```

#include <stdio.h>

int main(void)
{
    double x0 = 2.0;

```

```
double v0 = 5.0;
double acceleration = 3.0;
double time = 4.0;

double position =
    x0
    + v0 * time
    + 0.5 * acceleration * time * time;

printf("Initial position = %f m\n", x0);
printf("Initial velocity = %f m/s\n", v0);
printf("Acceleration      = %f m/s^2\n", acceleration);
printf("Time              = %f s\n", time);
printf("Final position    = %f m\n", position);

return 0;
}
```

Sekali lagi, sebelum mempercayai program kita sudah mempunyai prediksi:

$$x = 46 \text{ m.}$$

Jika program tidak memberikan nilai tersebut, kita harus memeriksa kembali programnya.

1.5 Program adalah representasi model

Ada satu gagasan penting yang perlu diperhatikan sejak awal. Komputer tidak mengetahui fisika! Ketika kita menulis

```
double voltage = current * resistance;
```

komputer tidak mengetahui bahwa kita sedang menggunakan Hukum Ohm. Komputer hanya menjalankan operasi numerik yang kita berikan. Demikian pula ketika kita menulis

```
double energy = 0.5 * mass * velocity * velocity;
```

komputer tidak memahami konsep energi kinetik.

Makna fisika berasal dari kita sebagai pembuat model dan program. Dengan kata lain,

komputer menjalankan instruksi, manusia menentukan maknanya.

Karena itu, kesalahan dalam model atau algoritma dapat menghasilkan program yang berjalan sempurna secara teknis, tetapi salah secara ilmiah.

Dalam komputasi sains dan rekayasa, kita perlu memeriksa setidaknya tiga hal:

1. apakah model fisis yang digunakan sudah tepat;
2. apakah algoritma merepresentasikan model tersebut dengan benar;
3. apakah program mengimplementasikan algoritma dengan benar.

Jika salah satu tahap tersebut bermasalah, hasil komputasi juga dapat salah.

1.5.1 Pentingnya satuan dalam komputasi fisika

Pemrograman tidak menghapus kebutuhan untuk memahami satuan. Misalkan kita menggunakan

$$v = 72 \text{ km/jam}$$

dalam persamaan energi kinetik, tetapi program menganggap nilai tersebut memiliki satuan m/s. Program dapat berjalan tanpa *error*, tetapi hasilnya salah secara fisika. Karena

$$72 \text{ km/jam} = 20 \text{ m/s},$$

memasukkan angka 72 ketika program mengharapkan satuan m/s akan menghasilkan kesalahan besar.

Dari sini, nama variabel, dokumentasi, dan pemahaman satuan sangat penting dalam program teknik. Pada contoh sederhana, kita dapat menambahkan komentar:

```
double velocity = 20.0; // m/s
```

Komentar tidak dijalankan komputer. Komentar membantu manusia memahami program. Dalam bahasa C, komentar satu baris dimulai dengan `//` sedangkan komentar beberapa baris dapat ditulis sebagai

```
/*  
    komentar  
    beberapa baris  
*/
```

Kita akan menggunakan komentar secukupnya untuk menjelaskan bagian program yang memang membutuhkan penjelasan.

1.5.2 Komputasi tidak menggantikan penalaran fisika

Salah satu kesalahan umum ketika mulai menggunakan komputer adalah menganggap angka yang keluar dari program sebagai jawaban yang benar. Padahal, komputer hanya melakukan apa yang kita instruksikan.

Misalkan sebuah program menghitung kecepatan benda dan menghasilkan

$$v = 3.7 \times 10^{12} \text{ m/s}.$$

Program mungkin saja berjalan tanpa *error*. Namun, hasil tersebut jauh lebih besar daripada kecepatan cahaya,

$$c \approx 3 \times 10^8 \text{ m/s}.$$

Contoh ini menunjukkan indikasi kuat bahwa model, satuan, input, atau program perlu diperiksa.

Dalam komputasi ilmiah selalu tanyakan:

Apakah hasil ini masuk akal secara fisika?

Pertanyaan tersebut sama pentingnya dengan pertanyaan

Apakah program berhasil dijalankan?

1.6 Perumusan algoritma

Perhatikan persamaan

$$P = VI.$$

Persamaan tersebut menyatakan hubungan matematis antara daya, tegangan, dan arus. Jika kita ingin membuat program untuk menghitung daya, kita perlu menetapkan secara lebih eksplisit:

```
Baca tegangan V
Baca arus I
Hitung P = V * I
Tampilkan P
```

Inilah bentuk awal algoritma.

Untuk persoalan sederhana, algoritma hampir sama langsungnya dengan persamaan. Namun, untuk persoalan yang lebih kompleks, kita harus memikirkan:

- ▶ urutan perhitungan;
- ▶ keputusan;
- ▶ pengulangan;
- ▶ penyimpanan data;
- ▶ validasi input;
- ▶ penanganan kondisi khusus;
- ▶ efisiensi.

Dalam materi mendatang, kita akan berfokus pada cara menuliskan algoritma sebelum menerjemahkannya ke program.

1.6.1 Proses dari bit menuju program ilmiah

Sekarang kita dapat melihat beberapa lapisan abstraksi dalam komputasi. Pada tingkat fisika perangkat keras, komputer terdiri atas sistem elektronik. Pada tingkat representasi data, kita berbicara tentang bit:

```
0 1 1 0 1 0 1 1
```

Pada tingkat bahasa mesin, bit-bit tersebut membentuk data dan instruksi. Di atasnya, *compiler* memungkinkan kita menulis bahasa tingkat tinggi seperti C:

```
energy = 0.5 * mass * velocity * velocity;
```

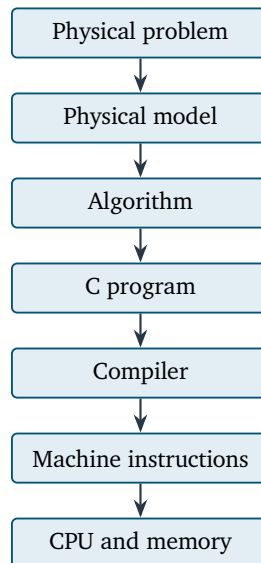
Di atas itu lagi terdapat algoritma:

```
baca massa  
baca kecepatan  
hitung energi kinetik  
tampilkan hasil
```

Pada tingkat paling dekat dengan pekerjaan seorang *engineer*, kita mempunyai model fisika:

$$E_k = \frac{1}{2}mv^2.$$

Kita dapat menggambarkan lapisan-lapisan tersebut sebagai berikut.



Seorang *programmer* tidak perlu selalu berpikir pada tingkat transistor atau instruksi mesin. Akan tetapi, memahami bahwa lapisan-lapisan tersebut ada membantu kita memahami mengapa tipe data mempunyai batas, mengapa bilangan real tidak selalu eksak, dan mengapa program perlu dikompilasi.

1.7 Cek pemahaman

Cobalah menjawab beberapa pertanyaan berikut tanpa melihat kembali catatan.

- ▶ Apa perbedaan antara algoritma dan program?
- ▶ Mengapa CPU tidak dapat langsung menjalankan source code C?
- ▶ Apa fungsi *compiler*?
- ▶ Apa perbedaan antara kode sumber (*source code*) dan program yang dapat dieksekusi (*executable*)?
- ▶ Berapa banyak pola berbeda yang dapat direpresentasikan menggunakan delapan bit?

- ▶ Mengapa *unsigned integer* delapan bit mempunyai rentang 0 sampai 255?
- ▶ Mengapa

$$13_{10} = 1101_2?$$
- ▶ Apa keuntungan menggunakan notasi heksadesimal dalam komputasi?
- ▶ Mengapa suatu integer mempunyai nilai maksimum?
- ▶ Apa yang dimaksud dengan *overflow*?
- ▶ Mengapa karakter dapat disimpan oleh komputer?
- ▶ Mengapa bilangan seperti 0.1 tidak selalu dapat direpresentasikan secara eksak?
- ▶ Apa perbedaan antara *precision* dan *range*?
- ▶ Mengapa program yang berhasil dikompilasi belum tentu memberikan hasil yang benar?
- ▶ Mengapa hasil program fisika perlu dibandingkan dengan prediksi atau perhitungan manual?

Jika sebagian besar pertanyaan tersebut dapat dijawab dengan baik, kita sudah mempunyai fondasi yang diperlukan untuk melanjutkan ke perancangan algoritma.

1.8 Latihan

1.8.1 Representasi biner

Konversikan bilangan berikut ke desimal:

$$1011_2, \quad 100101_2, \quad 11111111_2, \quad 10000000_2$$

Konversikan bilangan berikut ke biner:

$$7_{10}, \quad 18_{10}, \quad 25_{10}, \quad 42_{10}, \quad 100_{10}$$

1.8.2 Representasi heksadesimal

Ubah bilangan berikut dari biner menjadi heksadesimal:

$$1111_2, \quad 1010_2, \quad 11111111_2, \quad 10101111_2, \quad 1100101011110000_2$$

Kemudian ubah $3A_{16}$ dan FF_{16} ke bentuk desimal.

1.8.3 Rentang integer

Jika sebuah *unsigned integer* menggunakan 10 bit, tentukan:

- ▶ jumlah pola bit yang mungkin;
- ▶ nilai minimum;
- ▶ nilai maksimum.

Ulangi untuk *unsigned integer* 16 bit.

Kemudian, tentukan rentang *signed integer* 8 bit dan *signed integer* 16 bit dengan asumsi representasi *two's complement*.

1.8.4 Memahami *overflow*

Sebuah *unsigned integer* 8 bit memiliki nilai maksimum 255. Jelaskan mengapa nilai 256 tidak dapat direpresentasikan menggunakan delapan bit. Berapa banyak bit minimum yang diperlukan untuk merepresentasikan 256 sebagai *unsigned integer*?

1.8.5 Floating-point

Jelaskan dengan kalimat sendiri mengapa komputer tidak selalu dapat menyimpan nilai 0.1 secara eksak. Apa hubungan masalah tersebut dengan representasi

$$\frac{1}{3} = 0.333333 \dots$$

dalam sistem desimal? Mengapa perbandingan dua hasil floating-point menggunakan kesamaan eksak (misal $0.3 = 0.2 + 0.1$) dapat menimbulkan masalah?

1.8.6 Algoritma sederhana

Tuliskan algoritma dengan kalimat biasa untuk menghitung daya listrik dari

$$P = VI.$$

Tentukan dengan jelas yang mana

- ▶ input;
- ▶ proses;
- ▶ output.

Lakukan hal yang sama untuk persamaan gas ideal

$$PV = nRT$$

dengan tujuan mencari tekanan P .

1.8.7 Program C sederhana

Buat program `power.c` untuk menghitung daya listrik dari

$$P = VI.$$

Gunakan

$$V = 12.0 \text{ V},$$

dan

$$I = 2.5 \text{ A}.$$

Hitung terlebih dahulu hasilnya secara manual. Kemudian bandingkan dengan keluaran program. Kompilasi menggunakan

```
gcc -Wall -Wextra power.c -o power
```

Program yang baik pada tahap ini setidaknya harus dapat dikompilasi tanpa peringatan (*warning*).

1.8.8 Program gerak lurus

Buat program `motion.c` yang menghitung

$$x = x_0 + v_0 t + \frac{1}{2} a t^2.$$

Gunakan

$$\begin{aligned} x_0 &= 2.0 \text{ m}, \\ v_0 &= 5.0 \text{ m/s}, \\ a &= 3.0 \text{ m/s}^2, \end{aligned}$$

dan

$$t = 4.0 \text{ s}.$$

Program harus menampilkan nilai

- ▶ posisi awal;
- ▶ kecepatan awal;
- ▶ percepatan;
- ▶ waktu;
- ▶ posisi akhir.

Sebelum menjalankan program, hitung hasilnya secara manual. Setelah menjalankan program, periksa apakah kedua hasil tersebut sesuai.

1.9 Rangkuman

- ▶ Pada pertemuan ini kita memulai dari sebuah gagasan sederhana: komputer adalah alat untuk menjalankan prosedur komputasi. Persoalan fisika dapat diterjemahkan menjadi model matematis. Model tersebut kemudian diterjemahkan menjadi algoritma. Algoritma dapat diimplementasikan sebagai program.
- ▶ Hubungan tersebut dapat diringkas sebagai

masalah fisika → model matematis → algoritma → program → hasil

- ▶ Program C yang kita tulis masih harus diterjemahkan oleh *compiler* sebelum dapat dijalankan oleh komputer:

source code → compiler → machine code → execution.

- ▶ Pada tingkat representasi data, komputer menggunakan bit. Dengan n bit terdapat 2^n pola yang berbeda.
- ▶ Untuk *unsigned integer* n -bit,

$$0 \leq x \leq 2^n - 1.$$

- Untuk *signed integer two's complement*,

$$-2^{n-1} \leq x \leq 2^{n-1} - 1.$$

- Karena jumlah bit terbatas, representasi data juga terbatas. Hal ini menghasilkan konsep seperti *overflow* pada *integer* dan kesalahan pembulatan pada *floating-point*.
- Kita juga melihat bahwa keberhasilan program untuk dikompilasi dan dijalankan tidak menjamin bahwa hasilnya benar. Program perlu diperiksa menggunakan penalaran matematis, satuan, prediksi, dan pemahaman fisika.
- Pada tahap ini, tujuan kita bukan menguasai seluruh sintaks C. Kita baru membangun gambaran mengenai bagaimana masalah ilmiah dapat diterjemahkan menjadi komputasi.
- Pada pertemuan berikutnya kita akan memusatkan perhatian pada bagian

masalah $\longrightarrow$ algoritma.

- Kita akan mempelajari bagaimana sebuah prosedur dapat ditulis secara sistematis menggunakan kalimat berurut, diagram alir, dan pseudocode, terutama pseudocode matematis. Kita juga akan mulai membahas bagaimana menilai kebenaran serta kebutuhan komputasi suatu algoritma.